

Failure (a dead connection, and a real container)

— Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Read all three depths — they're not "wrong, less wrong, right," they're 3 real, valid ways to handle the same two problems (a broken database connection, and a real container), with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# db_failure_and_docker_practice.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()
conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    try:
        rows = conn.execute("SELECT * FROM runs").fetchall()
        return rows
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output:

```
$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail": "Something went wrong."}
```

No traceback reaches the client, and the server itself doesn't crash — `/health` still responds normally.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Expected output:

```
$ docker build -t my-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 my-api
$ curl localhost:8000/health
{"status":"ok"}
```

This satisfies the exercise's core requirements. It doesn't yet have a `.dockerignore`, and the failing connection is created once at import time rather than guarded on its own — both addressed below.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — guarding connection creation itself, plus a `.dockerignore`

```
# db_failure_and_docker_practice.py
import sqlite3
from fastapi import FastAPI, HTTPException

app = FastAPI()

try:
    conn = sqlite3.connect("/no/such/path/runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY, input
except sqlite3.OperationalError:
    conn = None

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    if conn is None:
        raise HTTPException(status_code=500, detail="Something went wrong.")
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output:

```
$ curl localhost:8000/health
{"status":"ok"}
$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}
```

Even though the database path is broken from the very first line, the app still starts and `/health` still works — only the routes that actually need the database fail, and fail cleanly.

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/*
*.pyc
.venv/
```

Expected output (`docker history my-api`): a list of layers, none of which contains `.env` or any file from `.git` — `.dockerignore` stops `COPY . .` from ever picking them up, so there's nothing to leak in the first place.

Approach 2 — a startup check that fails the container immediately if a required env var is missing

```
# db_failure_and_docker_practice.py
import os
import sqlite3
from fastapi import FastAPI, HTTPException

# Fails loudly at import time (container start) if this is missing –
# os.environ[...] raises KeyError immediately; os.getenv(...) would not.
OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]

app = FastAPI()

try:
    conn = sqlite3.connect("runs.db", check_same_thread=False)
    conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY, input
except sqlite3.OperationalError:
    conn = None

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
```

```

if conn is None:
    raise HTTPException(status_code=500, detail="Something went wrong.")
try:
    return conn.execute("SELECT * FROM runs").fetchall()
except Exception:
    raise HTTPException(status_code=500, detail="Something went wrong.")

```

Expected output with the required env var missing:

```

$ docker run -p 8000:8000 my-api
Traceback (most recent call last):
...
KeyError: 'OPENAI_API_KEY'
$ echo $?
1

```

The container exits immediately with a clear, specific error — not a silent hang, and not a container that looks "up" but can't actually do anything.

Difference from Basic: Approach 1 wraps connection *creation* itself in `try/except`, so a broken path fails the specific routes that need it instead of preventing the whole app from starting, and adds `.dockerignore` so `COPY . .` can never accidentally pick up `.env` or `.git`. Approach 2 adds a second, different kind of failure worth testing: a required setting that's simply missing at container startup — and shows why `os.environ[key]` (raises immediately) is the right choice over `os.getenv(key)` (returns `None` silently) for anything the app genuinely cannot function without.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a connection that dies mid-run, not just one that never worked

```
# db_failure_and_docker_practice.py
import os
import sqlite3
from fastapi import FastAPI, HTTPException

OPENAI_API_KEY = os.environ["OPENAI_API_KEY"]

app = FastAPI()
conn = sqlite3.connect("runs.db", check_same_thread=False)
conn.execute("CREATE TABLE IF NOT EXISTS runs (id INTEGER PRIMARY KEY, input TEXT)")

@app.get("/health")
def health():
    return {"status": "ok"}

@app.get("/runs")
def get_runs():
    try:
        return conn.execute("SELECT * FROM runs").fetchall()
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")

@app.post("/debug/kill-db")
def kill_db():
    """Testing-only route: simulates the connection dying mid-run."""
    conn.close()
    return {"status": "connection closed on purpose"}
```

Expected output, run in order against the same running process:

```
$ curl localhost:8000/health
{"status":"ok"}
$ curl localhost:8000/runs
[]
$ curl -X POST localhost:8000/debug/kill-db
{"status":"connection closed on purpose"}
```

```
$ curl -i localhost:8000/runs
HTTP/1.1 500 Internal Server Error
{"detail":"Something went wrong."}
```

The last call proves the `try/except` around the query catches a connection that was working and then stopped — not just one that was broken from the very first line, which is the more common way this actually happens in production (a disk fills up, a mounted volume gets unmounted, a network database drops the connection). Remove the `/debug/kill-db` route before shipping — it only exists to make this failure reproducible on demand.

Approach 2 — a multi-stage Dockerfile, and proving nothing leaked with `docker history`

```
# ---- build stage ----
FROM python:3.11-slim AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --user -r requirements.txt

# ---- final stage ----
FROM python:3.11-slim
WORKDIR /app
COPY --from=builder /root/.local /root/.local
COPY . .
ENV PATH=/root/.local/bin:$PATH
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# .dockerignore
.env
.git
__pycache__/
*.pyc
.venv/
*.db
```

Expected output:

```
$ docker build -t my-api .
$ docker run -p 8000:8000 -e OPENAI_API_KEY=sk-test-123 my-api
$ curl localhost:8000/health
{"status":"ok"}

$ docker history --no-trunc my-api | grep -i "sk-test-123"
```

(no output — the real key value appears nowhere in any layer, because it was only ever passed with `-e` at ``docker run`` time, never COPY-ed or baked in)

The multi-stage build here doesn't change what's safe to ship — it keeps the final image smaller by not carrying pip's build cache into the shipped image. The security proof is the same either way: `grep docker history --no-trunc` for the real secret value and confirm it finds nothing, for every image you ship.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate proves a connection that's broken from the start fails cleanly, and proves a missing required env var crashes the container immediately. Approach 1 goes further and proves a connection that was healthy can die mid-run and still be caught the same way — a more realistic production failure than "the path was always wrong." Approach 2 is a separate, complementary concern: a smaller shipped image via a multi-stage build, and an explicit, actually-run `docker history` check proving no secret survived in any layer — not just trusting that `.dockerignore` worked.

Which one should you actually write? Intermediate Approach 1 and 2 together are the minimum any real Dockerized API needs: guard connection creation, add `.dockerignore`, and fail loudly on a missing required env var. Add Advanced Approach 1's mid-run kill test once you've shipped this somewhere that isn't your laptop — a connection dying while the app is already running is the failure you'll actually see in production, not a path that was wrong from minute one. Reach for Approach 2's multi-stage build once image size or build speed genuinely matters (a slow CI pipeline, a slow deploy) — and run the `docker history --no-trunc | grep` check on every image before it ships, regardless of which Dockerfile shape you used.